

Design-Weighted LCA: Segments, Profiling, and Handoff

Institutional trust in Mexico: LAPOP AmericasBarometer 2023

Kevin Linares

2026-07-26

1 Purpose

Reads the fit saved by the modeling script, drafts or reads the segment labels, presents the measurement model, scores every respondent who answered enough items, profiles the segments on demographics against the design, and writes the handoff. A latent class is called a **segment** throughout; the statistical object is unchanged.

The order matters. The measurement model was estimated on item-complete cases in the modeling script. Scoring happens next and covers every respondent above `cfg$min_items`, which is a larger set. All domain estimates are then computed on that larger set, which is standard three-step practice and matters here because item nonresponse on a trust battery is not random: it skews older, less educated, and more rural, so restricting the domain estimates to complete responders would bias the composition in a predictable direction.

2 What the segment labeler is asked

The instrument, verbatim: the system prompt and one real user prompt built from this survey's own wording and probabilities. Each call carries one segment only, because joint prompts confuse near-neighbor segments. Rule 1 fences the survey context to referent resolution, so nothing outside the shown probabilities may be attributed to a segment.

```
cat("===== SYSTEM PROMPT =====\n")
```

```
===== SYSTEM PROMPT =====
```

```
cat(wrap_text(lca_persona()), "\n\n")
```

You are a senior survey methodologist who reads latent class analysis (LCA) measurement models. In this work each latent class is called a SEGMENT; that is a word-choice preference and the statistical object is unchanged. Each segment is described only by its item-response probabilities: for every survey item, the probability that a member of that segment gives each answer. A segment leans toward the answers with high probability. You interpret a segment strictly from these probabilities and the item wording, never from outside assumptions.

```
cat("===== USER PROMPT (segment 1) =====\n")
```

```
===== USER PROMPT (segment 1) =====
```

```
cat(wrap_text(prompt_segment_label(fit_w, 1, dictionary, cfg$items,
                                   cfg$survey_context)), "\n")
```

SURVEY CONTEXT

These items come from the 2023 AmericasBarometer survey of Mexico, conducted by the LAPOP Lab at Vanderbilt University. The AmericasBarometer is a comparative public opinion study of democratic attitudes and governance across the Americas, fielded to national probability samples of voting-age adults.

The battery analyzed here measures trust in national institutions: respondents rate, for each institution, how much they trust it on a seven-point scale anchored at 1 (not at all) and 7 (a lot). The segments summarize patterns of institutional trust across these items.

ONE SEGMENT FROM A LATENT CLASS ANALYSIS (LCA) MEASUREMENT MODEL

SEGMENT 1 (estimated prevalence 34%):

justice_system "Confianza en el sistema de justicia"

P(Nada)=0.00, P(2)=0.04, P(3)=0.16, P(4)=0.45, P(5)=0.28, P(6)=0.06, P(Mucho)=0.01

electoral_tribunal "Confianza en el Tribunal Supremo Electoral"

P(Nada)=0.03, P(2)=0.06, P(3)=0.12, P(4)=0.27, P(5)=0.28, P(6)=0.17, P(Mucho)=0.08

armed_forces "Confianza en las fuerzas armadas"

P(Nada)=0.02, P(2)=0.03, P(3)=0.07, P(4)=0.17, P(5)=0.27, P(6)=0.25, P(Mucho)=0.20

legislature "Confianza en la legislatura"

P(Nada)=0.02, P(2)=0.03, P(3)=0.11, P(4)=0.38, P(5)=0.30, P(6)=0.13, P(Mucho)=0.04

public_ministry "Confianza en el Ministerio Público"

P(Nada)=0.01, P(2)=0.03, P(3)=0.10, P(4)=0.36, P(5)=0.35, P(6)=0.11, P(Mucho)=0.04

police "Confianza en la policía nacional"

P(Nada)=0.08, P(2)=0.18, P(3)=0.28, P(4)=0.26, P(5)=0.14, P(6)=0.05, P(Mucho)=0.02

auditor "Confianza en la Auditoría Superior de la Federación"

P(Nada)=0.01, P(2)=0.02, P(3)=0.14, P(4)=0.41, P(5)=0.30, P(6)=0.10, P(Mucho)=0.02

political_parties "Confianza en los partidos políticos"

P(Nada)=0.11, P(2)=0.19, P(3)=0.27, P(4)=0.29, P(5)=0.11, P(6)=0.03, P(Mucho)=0.01

president "Confianza en el presidente"

P(Nada)=0.04, P(2)=0.06, P(3)=0.07, P(4)=0.17, P(5)=0.19, P(6)=0.21, P(Mucho)=0.27

supreme_court "Confianza en el tribunal supremo"

P(Nada)=0.01, P(2)=0.04, P(3)=0.12, P(4)=0.31, P(5)=0.36, P(6)=0.14, P(Mucho)=0.04
municipality "Confianza en su municipalidad"
P(Nada)=0.01, P(2)=0.04, P(3)=0.14, P(4)=0.36, P(5)=0.33, P(6)=0.09, P(Mucho)=0.02
media "Confianza en los medios de comunicación"
P(Nada)=0.05, P(2)=0.11, P(3)=0.21, P(4)=0.26, P(5)=0.25, P(6)=0.08, P(Mucho)=0.04
elections "Confianza en las elecciones"
P(Nada)=0.02, P(2)=0.05, P(3)=0.21, P(4)=0.30, P(5)=0.26, P(6)=0.13, P(Mucho)=0.03

TASK

Read this single segment and return: a short DRAFT label (2 to 5 words) for an analyst to refine, and a one or two sentence factual description anchored to its high-probability answers.

RULES:

1. Use only the response probabilities and item wording shown. Survey context only clarifies what the items refer to; attribute nothing to the segment that the probabilities do not show.
 2. Anchor every statement to the high-probability answers of this segment.
 3. If the profile is diffuse (no clear high-probability answers), say so.
 4. Return only valid JSON: no prose before or after, no markdown fences.
- JSON (one object): {"label": "...", "description": "..."}

3 Naming the segments

segment_labels.csv in the output directory is used when it exists. Otherwise the model drafts once per segment and writes it, with any harmonizer edit recorded in Label_draft. Editing that file is taking over the naming. The labels below are drafts to verify against the profile heatmap.

```
segment_labels <- get_segment_labels(fit_w, dictionary, cfg$items, cfg)

lca_table(segment_labels, widths = c("2em", "11em", "28em"),
  caption = "Segment labels and descriptions in use. Verify each against its response profile")
```

Table 1: Segment labels and descriptions in use. Verify each against its response profile before quoting.

K	Label	Description
1	Guarded Political Distrust	This segment tends toward moderate trust (rating of 4) in most national institutions, including the justice system, auditor, and supreme court. However, they express notably lower trust in political parties and the police, with higher probabilities of selecting 'not at all' or low ratings (1-3) for these institutions.
2	Moderate, Reserved Trust	This segment generally expresses moderate trust in most national institutions, with response probabilities peaking around 5 or 6 on the seven-point scale. While outright distrust (ratings of 1 or 2) is rare across institutions, strong trust (ratings of 7) is also uncommon, suggesting a cautious or reserved level of confidence.

Table 1: Segment labels and descriptions in use. Verify each against its response profile before quoting. (*continued*)

K	Label	Description
3	Widespread Institutional Distrust	This segment is characterized by low trust across nearly all national institutions, with probabilities exceeding 0.60 for 'not at all' (Nada) for most. Political parties stand out as particularly distrusted, with an 86% probability of a 'not at all' response.
4	Cautious, Moderate Trust	This segment generally expresses moderate levels of trust in most national institutions, frequently selecting '3' as their response. However, they exhibit notably lower trust in the police and political parties, with a high probability of responding 'Nada' (not at all) for these institutions.
5	High Presidential Trust	This segment expresses strong trust in the president (82% give a 'lot' of trust) and generally high trust in the justice system, electoral tribunal, armed forces, public ministry, and supreme court (all above 70% 'a lot'). While trust is lower in other institutions, this segment avoids expressing 'not at all' distrust across most items.

```
lab_lev <- str_wrap(paste0("S", segment_labels$K, ": ", segment_labels$Label), 22)
lab_of <- function(k) factor(lab_lev[k], levels = lab_lev)
lab_line <- function(k) paste0("S", k, ": ", segment_labels$Label[k])
lab_short <- function(k) paste0("S", k)
```

4 The measurement model

Intervals here are the JK_n intervals computed in the modeling script on the estimation sample. They describe the measurement model, not the scored sample.

```
cat(sprintf(paste0("VALIDATION (unit weights, pseudo-EM vs poLCA): max |pi gap| = %.5f, ",
  "max |rho gap| = %.5f -> %s\n"),
  fit$validation$max_pi_gap, fit$validation$max_rho_gap,
  if (fit$validation$match) "MATCH" else "MISMATCH, investigate"))
```

```
VALIDATION (unit weights, pseudo-EM vs poLCA): max |pi gap| = 0.00000, max |rho gap| = 0.00000
```

```
pi_ci <- ci_tbl |>
  filter(kind == "pi") |>
  transmute(K = segment, est = estimate, lo, hi)

pi_ci |>
  mutate(Segment = lab_line(K),
    `Weighted prevalence [95% CI]` = sprintf("%.3f [%.3f, %.3f]", est, lo, hi),
    `poLCA (unweighted)` = sprintf("%.3f", pl_aligned$pi[K]),
    Gap = sprintf("%.3f", est - pl_aligned$pi[K])) |>
  select(Segment:Gap) |>
```

```
lca_table(align = "lrrr", widths = c("13em", "13em", "9em", "5em"),
          caption = "Segment prevalences: design-weighted estimates with JKn intervals, the u
```

Table 2: Segment prevalences: design-weighted estimates with JKn intervals, the unweighted poLCA benchmark, and the gap. Read the gap against the unequal-weighting effect reported in the modeling script; when that is near 1 the gap is noise rather than correction.

Segment	Weighted prevalence [95% CI]	poLCA (unweighted)	Gap
S1: Guarded Political Distrust	0.341 [0.300, 0.384]	0.341	+0.000
S2: Moderate, Reserved Trust	0.207 [0.171, 0.249]	0.207	+0.000
S3: Widespread Institutional Distrust	0.107 [0.084, 0.135]	0.107	-0.000
S4: Cautious, Moderate Trust	0.234 [0.192, 0.281]	0.234	-0.000
S5: High Presidential Trust	0.112 [0.093, 0.134]	0.112	+0.000

```
pi_ci |>
  mutate(Segment = lab_of(K)) |>
  ggplot(aes(Segment, est)) +
  geom_pointrange(aes(ymin = lo, ymax = hi), linewidth = 0.5) +
  geom_point(aes(y = pl_aligned$pi[K]), shape = 5, size = 2.6) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(x = NULL, y = "Prevalence") +
  theme_lca() +
  theme(axis.text.x = element_text(angle = 20, hjust = 1))
```

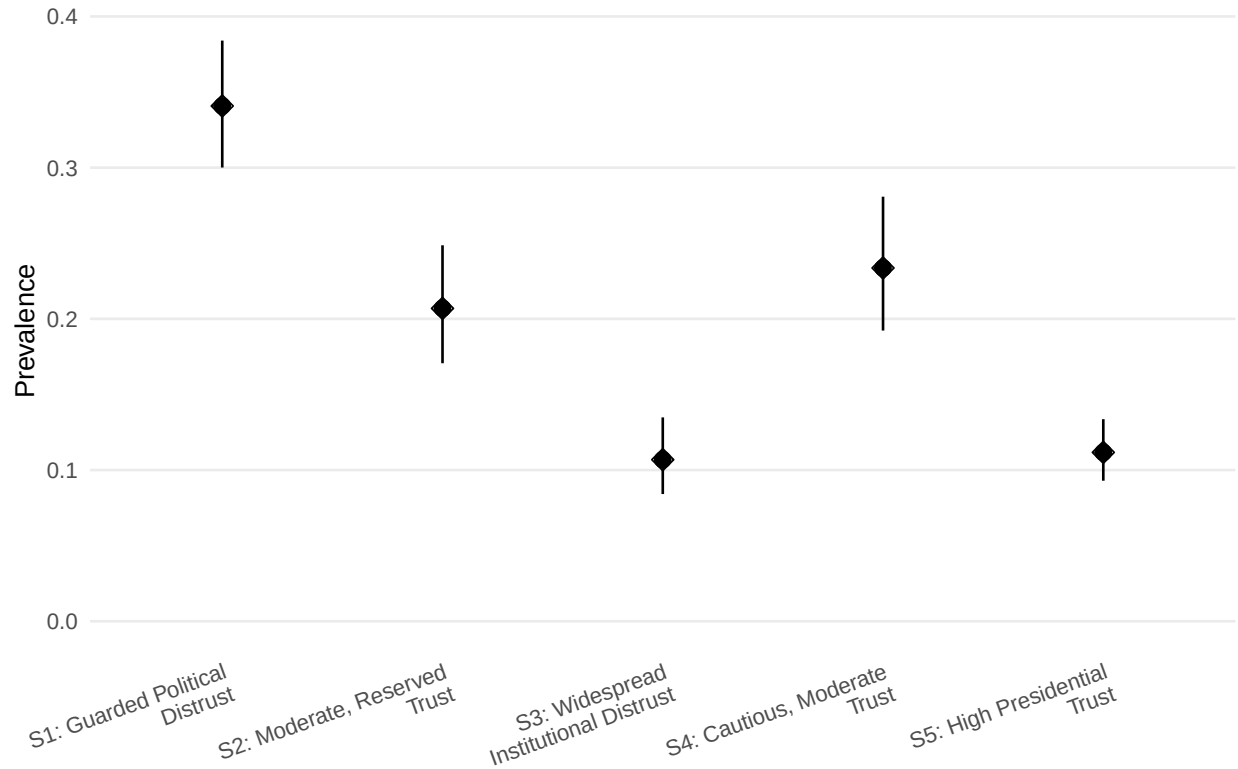


Figure 1: Weighted segment prevalences with 95% JK intervals against the unweighted poLCA benchmark (open diamonds).

```
ci_tbl |>
  filter(kind == "rho") |>
  mutate(Segment = lab_of(segment), item = factor(item, levels = cfg$items)) |>
  ggplot(aes(factor(category), item, fill = estimate)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", estimate)), size = 2.1) +
  scale_fill_viridis_c(name = "P", limits = c(0, 1)) +
  facet_wrap(~ Segment, ncol = min(3, K_star)) +
  labs(x = "Response category", y = NULL) +
  theme_lca() +
  theme(axis.text.y = element_text(size = 7))
```

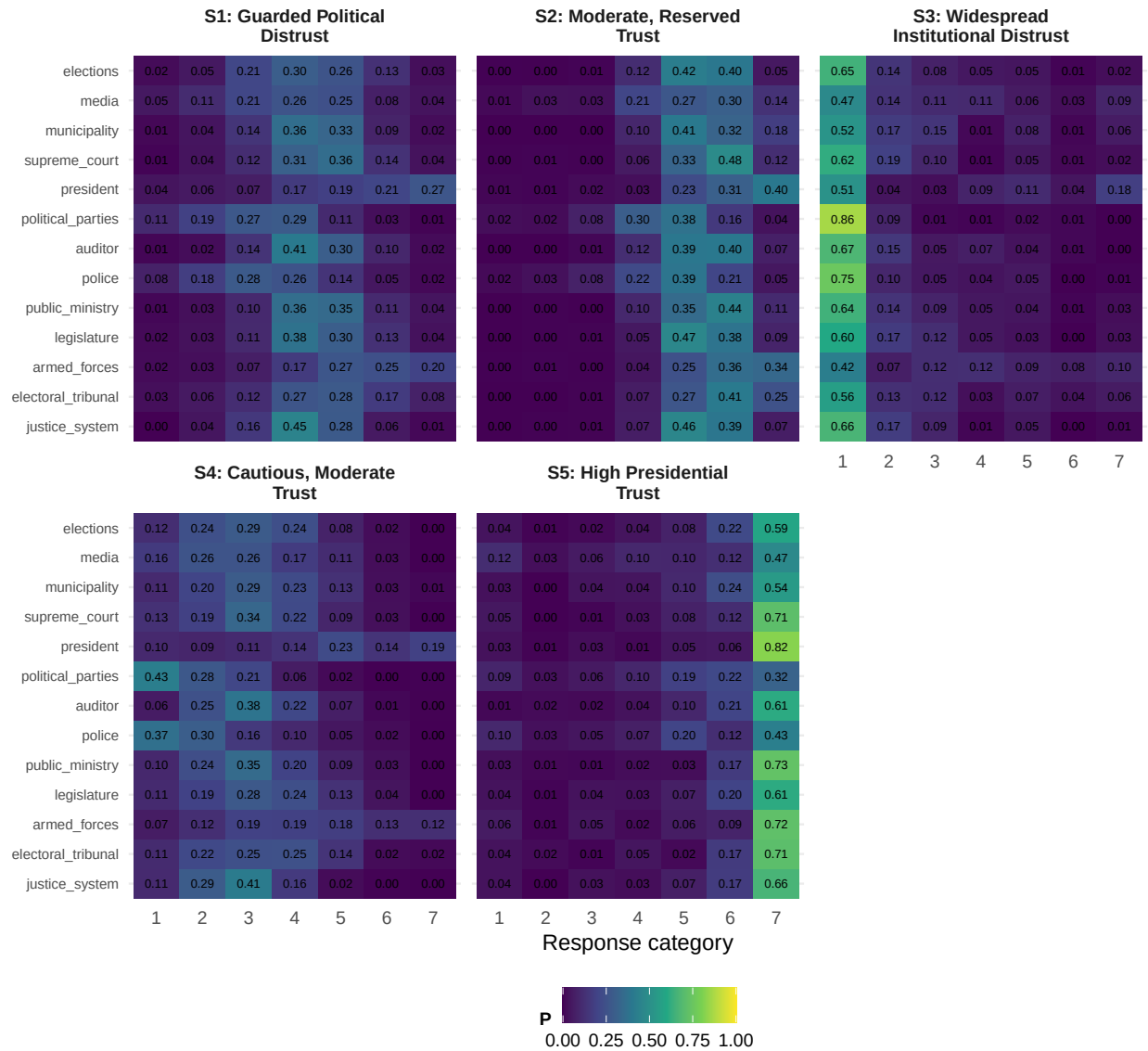


Figure 2: $P(\text{response} \mid \text{segment})$ for every item. Panel headers carry the drafted labels; verify each label against its panel before quoting it.

5 Scoring segment membership

Local independence lets a missing item drop out of the within-segment product, so a respondent who answered `cfg$min_items` or more is scorable even though they were not in the estimation sample. Everything downstream runs on the scored frame.

```
pred <- predict_segments(survey_dat_full, fit_w, cfg$items, cfg$min_items)
survey_scored <- bind_cols(survey_dat_full, pred)

scored <- survey_scored |>
```

```

filter(!is.na(segment)) |>
mutate(segment = factor(segment, levels = seq_len(K_star)))

design <- build_rep_design(scored, cfg)
des <- design$des
rep_des <- design$rep_des
design_df <- degf(rep_des)
crit <- qt(0.975, df = design_df)

cat(sprintf(paste0("Estimation sample: %d item-complete cases.\n",
  "Scored sample: %d of %d respondents (floor: %d items), ",
  "a gain of %d cases for the domain estimates.\n",
  "Domain design: %d replicates, %d degrees of freedom.\n"),
  nrow(fit$dat_prepared), nrow(scored), nrow(survey_scored),
  cfg$min_items, nrow(scored) - nrow(fit$dat_prepared),
  ncol(rep_des$repweights), design_df))

```

Estimation sample: 1454 item-complete cases.

Scored sample: 1618 of 1622 respondents (floor: 7 items), a gain of 164 cases for the domain estimates.

Domain design: 130 replicates, 126 degrees of freedom.

```

# Overall segment prevalence on the scored frame, with the design effect reported
# here rather than per demographic level. deff is the design variance over the
# variance a simple random sample of the same size would give, computed directly
# so it does not depend on a finite-population correction; with self-weighting
# data the sum of weights equals n and survey's deff = TRUE returns Inf.
overall <- svymean(~ segment, rep_des)

overall_tbl <- tibble(K = seq_len(K_star),
  p = as.numeric(coef(overall)),
  se = as.numeric(SE(overall))) |>
  mutate(lo = pmax(0, p - crit * se), hi = pmin(1, p + crit * se),
    deff = se^2 / (p * (1 - p) / nrow(scored)))

overall_tbl |>
  transmute(Segment = lab_line(K),
    Prevalence = sprintf("%.3f (%.3f)", p, se),
    `95% CI` = sprintf("[%.3f, %.3f]", lo, hi),
    Deff = sprintf("%.2f", deff)) |>
  lca_table(align = "lrrr", widths = c("14em", "10em", "11em", "5em"),
    caption = "Overall segment prevalence on the scored frame, with JK standard errors")

```

Table 3: Overall segment prevalence on the scored frame, with JK_n standard errors and design effects. Deff above 1 means clustering and weighting cost effective sample size relative to a simple random sample of the same size; below 1 means stratification bought some back.

Segment	Prevalence	95% CI	Deff
S1: Guarded Political Distrust	0.347 (0.013)	[0.322, 0.372]	1.14
S2: Moderate, Reserved Trust	0.205 (0.012)	[0.182, 0.227]	1.33
S3: Widespread Institutional Distrust	0.112 (0.008)	[0.096, 0.129]	1.12
S4: Cautious, Moderate Trust	0.227 (0.012)	[0.203, 0.251]	1.35
S5: High Presidential Trust	0.109 (0.008)	[0.094, 0.125]	0.99

```
scored |>
  group_by(n_items_answered) |>
  summarise(n = n(), mean_max_posterior = round(mean(max_posterior), 3),
            .groups = "drop") |>
  lca_table(align = "rrr", widths = c("10em", "6em", "12em"),
            col.names = c("Items answered", "n", "Mean max posterior"),
            caption = "Assignment quality by number of items answered. Respondents scored from
```

Table 4: Assignment quality by number of items answered. Respondents scored from fewer items carry flatter posteriors and so more classification error, which the domain estimates below pool over. If the mean falls sharply at the low end, raise `cfg$min_items`.

Items answered	n	Mean max posterior
7	3	0.718
8	5	0.889
9	5	0.891
10	22	0.818
11	25	0.901
12	104	0.903
13	1454	0.940

6 What the design costs

The design effect is the ratio of the variance under this design to the variance a simple random sample of the same size would give. A design effect of 2 means the design cost half the effective sample size; below 1 means it bought some. The table reports it twice: once for the full design and once for a clustering-only design that ignores the strata, so the two sources can be separated.

Three patterns are worth reading for.

A design effect near the mean cluster size is the ceiling, and it is a specification check. A variable that is constant within a sampling unit has an intraclass correlation of 1, so its design effect equals the mean cluster size exactly. Any variable measured at the cluster level rather than the respondent level should land there. If it does, the cluster identifier is specified correctly and the replicate variance is computing what it should. If a respondent-level variable lands there, that

variable is effectively a cluster attribute and its precision claims are much weaker than its sample size suggests.

A design effect below 1 means the variance is smaller than random sampling would produce, which requires the composition of each cluster to be controlled rather than sampled. That is the signature of a quota at the final selection stage: if interviewers must fill a fixed number of respondents in some category per sampling point, the between-cluster variance for that category is suppressed. Precision on those variables is a fieldwork artifact, not information, so read their standard errors with that in mind and do not use them to demonstrate design-based variance estimation.

If the two design effect columns agree, stratification is contributing nothing and clustering accounts for all of the difference from simple random sampling. Coarse strata relative to the number of clusters will do this.

```
srs <- svydesign(ids = ~1, weights = reformulate(cfg$weight), data = scored)
clus <- svydesign(ids = reformulate(cfg$psu), weights = reformulate(cfg$weight),
  data = scored)

psu_n <- n_distinct(scored[[cfg$psu]])

deff_tbl <- map(cfg$aux, function(v) {
  f <- reformulate(v)
  full <- svymean(f, rep_des, na.rm = TRUE)
  tibble(variable = v,
    level = str_remove(names(coef(full)), paste0("^", v)),
    se_design = as.numeric(SE(full)),
    se_clus = as.numeric(SE(svymean(f, clus, na.rm = TRUE))),
    se_srs = as.numeric(SE(svymean(f, srs, na.rm = TRUE))))
}) |>
  list_rbind() |>
  mutate(deff = (se_design / se_srs)^2, deff_nostrata = (se_clus / se_srs)^2)

cat(sprintf(paste0("%d clusters, mean cluster size %.1f. A variable constant ",
  "within a cluster has design effect %.1f; that is the ceiling.\n"),
  psu_n, nrow(scored) / psu_n, nrow(scored) / psu_n))
```

130 clusters, mean cluster size 12.4. A variable constant within a cluster has design effect 12.4.

```
deff_tbl |>
  transmute(Variable = variable, Level = level,
    `SE design` = sprintf("%.4f", se_design),
    `SE SRS` = sprintf("%.4f", se_srs),
    Deff = sprintf("%.2f", deff),
    `Deff no strata` = sprintf("%.2f", deff_nostrata)) |>
  lca_table(align = "llrrrr",
    widths = c("8em", "9em", "7em", "7em", "4em", "6em"),
    caption = "Design effects for the profiling variables. SE design is the JKn standard error")
```

Table 5: Design effects for the profiling variables. SE design is the JK_n standard error, SE SRS the standard error a simple random sample of the same size would give. Deff is their squared ratio; Deff no strata drops the stratification and keeps the clustering, so a match between the two columns means stratification contributes nothing.

Variable	Level	SE design	SE SRS	Deff	Deff no strata
age_cat	18-29	0.0053	0.0116	0.21	0.22
age_cat	30-44	0.0090	0.0107	0.70	0.72
age_cat	45-59	0.0106	0.0110	0.93	0.93
age_cat	60+	0.0107	0.0093	1.34	1.33
sex	Male	0.0058	0.0124	0.22	0.23
sex	Female	0.0058	0.0124	0.22	0.23
education	Secondary	0.0145	0.0123	1.40	1.49
education	None	0.0040	0.0038	1.12	1.14
education	Primary	0.0124	0.0100	1.55	1.54
education	Tertiary	0.0151	0.0098	2.37	2.41
urban	Urban	0.0354	0.0100	12.58	12.93
urban	Rural	0.0354	0.0100	12.58	12.93
employment	Employed	0.0125	0.0124	1.01	1.02
employment	Unemployed	0.0051	0.0052	0.98	0.99
employment	Not in labor force	0.0044	0.0043	1.03	1.01
employment	Student	0.0062	0.0059	1.11	1.09
employment	Homemaker	0.0116	0.0111	1.09	1.12
employment	Retired	0.0057	0.0058	0.96	0.98

7 Segment composition by demographics

One page per demographic: its marginal distribution, the segment prevalences within each of its levels, and a figure. Everything uses the scored frame.

Design-based applies the sampling weights and takes its variance from the JK_n replicates. It uses hard modal assignment, so it is attenuated: differences between demographic levels are understated by roughly the classification error rate. **BCH-corrected** additionally reweights modal assignment by the inverse of the design-weighted classification error matrix (Bolck, Croon and Hagenaars 2004; Vermunt 2010), removing that attenuation at the cost of wider intervals. Step-1 parameter uncertainty is not propagated, the standard treatment.

The gap between them is what classification error costs. Attenuation always biases toward the marginal, so it always understates group differences: a difference that survives under modal assignment is real, but a null under modal assignment may be a real difference that attenuation erased. Read the design-based numbers as primary and the corrected numbers when the claim is that groups do not differ.

```
# 1. marginal distribution of each demographic, among those who answered it.
# The design effect is reported once for the overall estimates in the scoring
# section, not per level.
marginals <- map(cfg$aux, function(v) {
  m <- svymean(reformulate(v), rep_des, na.rm = TRUE)
  raw <- scored |>
    filter(!is.na(.data[[v]])) |>
    count(.data[[v]], name = "n") |>
```

```

    set_names(c("level", "n"))
  tibble(variable = v,
    level = str_remove(names(coef(m)), paste0("^", v)),
    weighted = as.numeric(coef(m)),
    se = as.numeric(SE(m))) |>
  left_join(mutate(raw, level = as.character(level)), by = "level")
}) |>
list_rbind()

# 2. segment prevalence within each level, straight from the survey package.
# svyby splits the design by the demographic and runs svymean(~segment) inside
# each level. Columns are read by position (by-variable, then K estimates, then
# K SEs) because svyby's naming for a factor outcome varies by version.
domain_marginals <- map(cfg$aux, function(v) {
  sb <- as.data.frame(svyby(~ segment, reformulate(v), rep_des, svymean,
    na.rm = TRUE))
  vals <- sb[, -1, drop = FALSE]
  stopifnot(ncol(vals) == 2 * K_star)
  tibble(variable = v,
    level = rep(as.character(sb[[1]]), times = K_star),
    segment = rep(seq_len(K_star), each = nrow(sb)),
    p = as.numeric(as.matrix(vals[, seq_len(K_star)])),
    se = as.numeric(as.matrix(vals[, K_star + seq_len(K_star)])))
}) |>
list_rbind() |>
filter(!is.na(level)) |>
mutate(lo = pmax(0, p - crit * se), hi = pmin(1, p + crit * se))

# 3. BCH-corrected version of the same quantity. Posteriors and modal assignment
# are fixed at the scored values; the error matrix and correction weights are
# rebuilt inside every replicate, so the intervals carry the design variance of
# the classification table and the cross-tab together.
dom_meta <- map(cfg$aux, function(v)
  expand_grid(plevel = levels(scored[[v]]), segment = seq_len(K_star)) |>
  mutate(pvar = v) |>
list_rbind() |>
select(pvar, plevel, segment) |>
mutate(idx = row_number())

P_fixed <- as.matrix(scored[, post_cols, drop = FALSE])
modal_fixed <- as.integer(scored$segment)

domain_theta <- function(w_rep) {
  wU <- w_rep * bch_weights(P_fixed, modal_fixed, w_rep)
  unlist(map(cfg$aux, function(v) {
    # indicator matrix with missing demographics contributing to no level
    M <- outer(as.character(scored[[v]]), levels(scored[[v]]), "==" ) + 0

```

```

    M[is.na(M)] <- 0
    as.vector(t(crossprod(M, wU) / as.numeric(crossprod(M, w_rep))))
  }, use.names = FALSE)
}

dom_est <- domain_theta(weights(des, "sampling"))
dom_V <- replicate_variance(rep_des, domain_theta, dom_est)
stopifnot(length(dom_est) == nrow(dom_meta))

dom_tbl <- dom_meta |>
  mutate(p = as.numeric(dom_est), se = sqrt(diag(dom_V)),
         inside = p > 0 & p < 1,
         p_c = pmin(pmax(p, 1e-6), 1 - 1e-6),
         gse = se / (p_c * (1 - p_c)),
         lo = if_else(inside, plogis(qlogis(p_c) - crit * gse), pmax(0, p - crit * se)),
         hi = if_else(inside, plogis(qlogis(p_c) + crit * gse), pmin(1, p + crit * se))) |>
  select(pvar, plevel, segment, p, se, lo, hi)

# Both estimators in one long frame, uncorrected then corrected.
est_levels <- c("Design-based", "BCH-corrected")

domain_all <- bind_rows(
  transmute(domain_marginals, variable, level, segment, p, se, lo, hi,
            estimator = "Design-based"),
  transmute(dom_tbl, variable = pvar, level = as.character(plevel), segment,
            p, se, lo, hi, estimator = "BCH-corrected")) |>
  mutate(estimator = factor(estimator, levels = est_levels))

walk(cfg$aux, function(v) {
  tag <- str_replace_all(v, "_", "-")
  cat("\n\n##", v, "\n\n")

  marginals |>
    filter(variable == v) |>
    transmute(Level = level, n, Share = sprintf("%.3f (%.3f)", weighted, se)) |>
    lca_table(align = "lrr", widths = c("12em", "4em", "12em"),
              label = paste0("comp-", tag),
              caption = paste0("Composition by ", v,
                                ". Share is the design-weighted proportion among ",
                                "respondents who answered the item, with its JKn ",
                                "standard error.))" |>

    cat(sep = "\n")
  cat("\n\n")

  domain_all |>
    filter(variable == v) |>
    transmute(Level = level, Estimator = as.character(estimator),

```

```

      Segment = lab_short(segment), cell = sprintf("%.3f", p)) |>
pivot_wider(names_from = Segment, values_from = cell) |>
arrange(Level, match(Estimator, est_levels)) |>
lca_table(align = str_c("ll", strrep("r", K_star)),
          widths = c("9em", "9em", rep("4em", K_star)),
          label = paste0("est-", tag),
          caption = paste0("Segment prevalence within each level of ", v,
                           ", uncorrected and BCH-corrected. Rows sum to one ",
                           "across segments. Intervals are in the figure below ",
                           "and the exported CSV.")) |>

cat(sep = "\n")
cat("\n\n")

p <- domain_all |>
  filter(variable == v) |>
  mutate(Segment = lab_of(segment)) |>
  ggplot(aes(p, level, color = estimator, shape = estimator)) +
  geom_pointrange(aes(xmin = lo, xmax = hi), size = 0.3, linewidth = 0.4,
                 position = position_dodge(width = 0.7)) +
  facet_wrap(~ Segment, ncol = min(3, K_star)) +
  scale_color_viridis_d(begin = 0.1, end = 0.75, direction = -1) +
  scale_shape_manual(values = c(`Design-based` = 16, `BCH-corrected` = 5)) +
  scale_x_continuous(limits = c(0, NA)) +
  labs(x = "Prevalence of segment membership", y = NULL, color = NULL,
       shape = NULL,
       caption = "95% intervals from JKn replicate variance and the design t. Filled circles")
  theme_lca()
print(p)
cat("\n\n{{< pagebreak >}}\n\n")
})

```

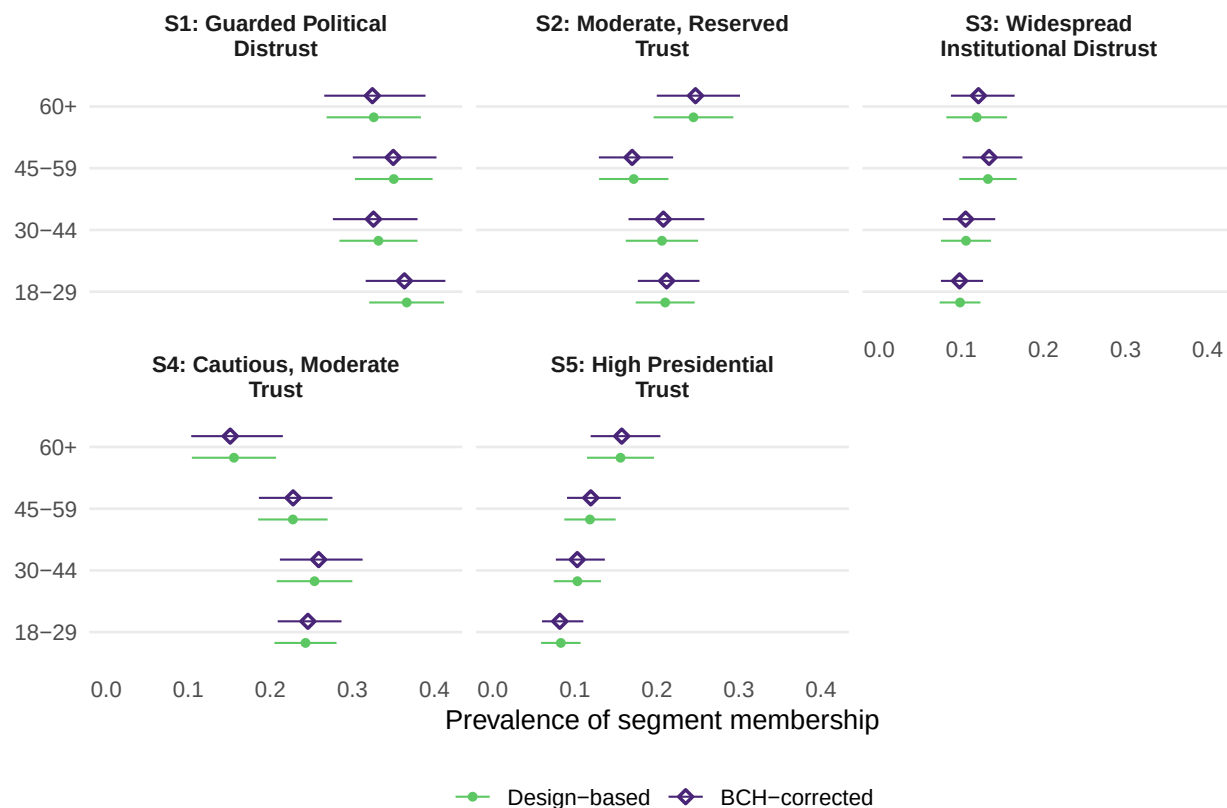
7.1 age_cat

Table 6: Composition by age_cat. Share is the design-weighted proportion among respondents who answered the item, with its JK_n standard error.

Level	n	Share
18-29	519	0.321 (0.005)
30-44	398	0.246 (0.009)
45-59	431	0.266 (0.011)
60+	270	0.167 (0.011)

Table 7: Segment prevalence within each level of age_cat, uncorrected and BCH-corrected. Rows sum to one across segments. Intervals are in the figure below and the exported CSV.

Level	Estimator	S1	S2	S3	S4	S5
18-29	Design-based	0.366	0.210	0.098	0.243	0.083
18-29	BCH-corrected	0.363	0.212	0.098	0.246	0.082
30-44	Design-based	0.332	0.206	0.106	0.254	0.103
30-44	BCH-corrected	0.326	0.208	0.105	0.259	0.103
45-59	Design-based	0.350	0.172	0.132	0.227	0.118
45-59	BCH-corrected	0.350	0.170	0.134	0.228	0.119
60+	Design-based	0.326	0.244	0.119	0.156	0.156
60+	BCH-corrected	0.324	0.247	0.121	0.151	0.157



95% intervals from JK_n replicate variance and the design t. Filled circles: design-based. Open diamonds: BCH-corrected

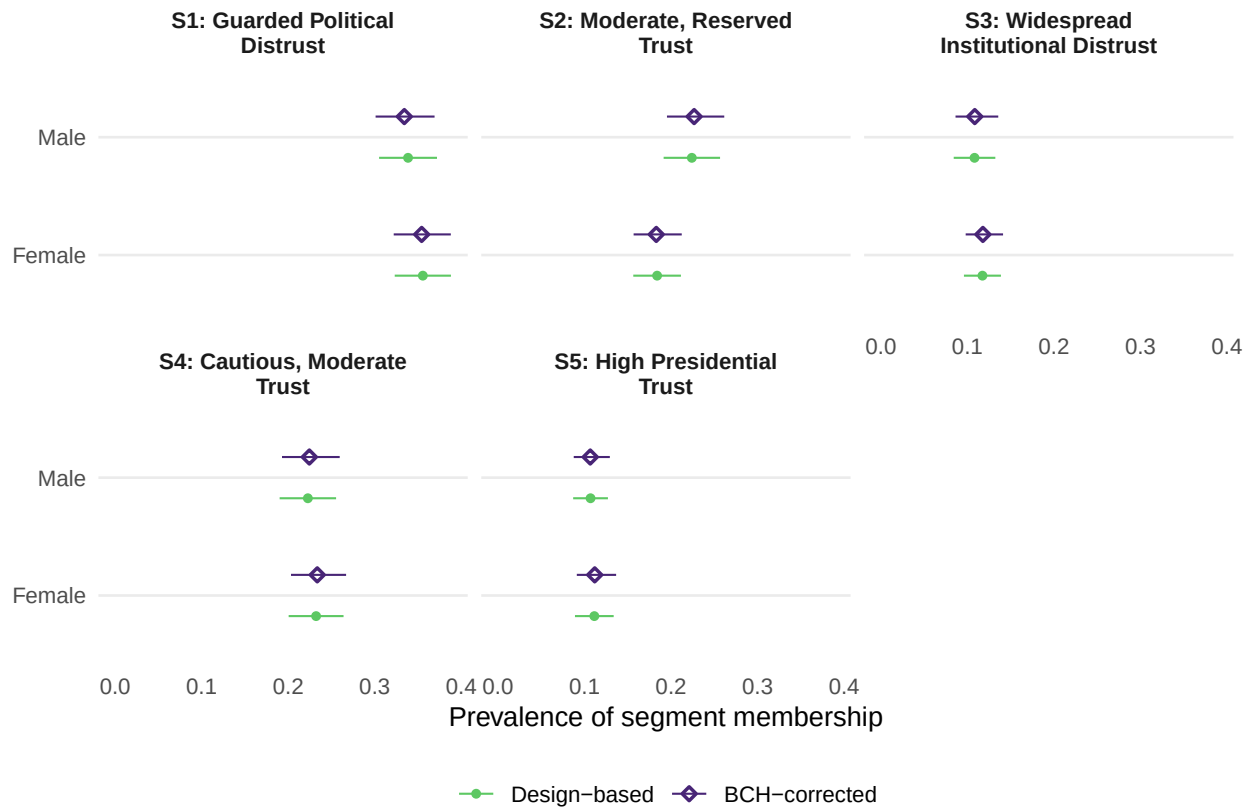
7.2 sex

Table 8: Composition by sex. Share is the design-weighted proportion among respondents who answered the item, with its JKn standard error.

Level	n	Share
Male	786	0.487 (0.006)
Female	827	0.513 (0.006)

Table 9: Segment prevalence within each level of sex, uncorrected and BCH-corrected. Rows sum to one across segments. Intervals are in the figure below and the exported CSV.

Level	Estimator	S1	S2	S3	S4	S5
Female	Design-based	0.356	0.184	0.117	0.232	0.111
Female	BCH-corrected	0.354	0.183	0.118	0.233	0.112
Male	Design-based	0.338	0.224	0.108	0.223	0.107
Male	BCH-corrected	0.334	0.227	0.108	0.224	0.107



95% intervals from JKn replicate variance and the design t. Filled circles: design-based. Open diamonds: BCH-corrected

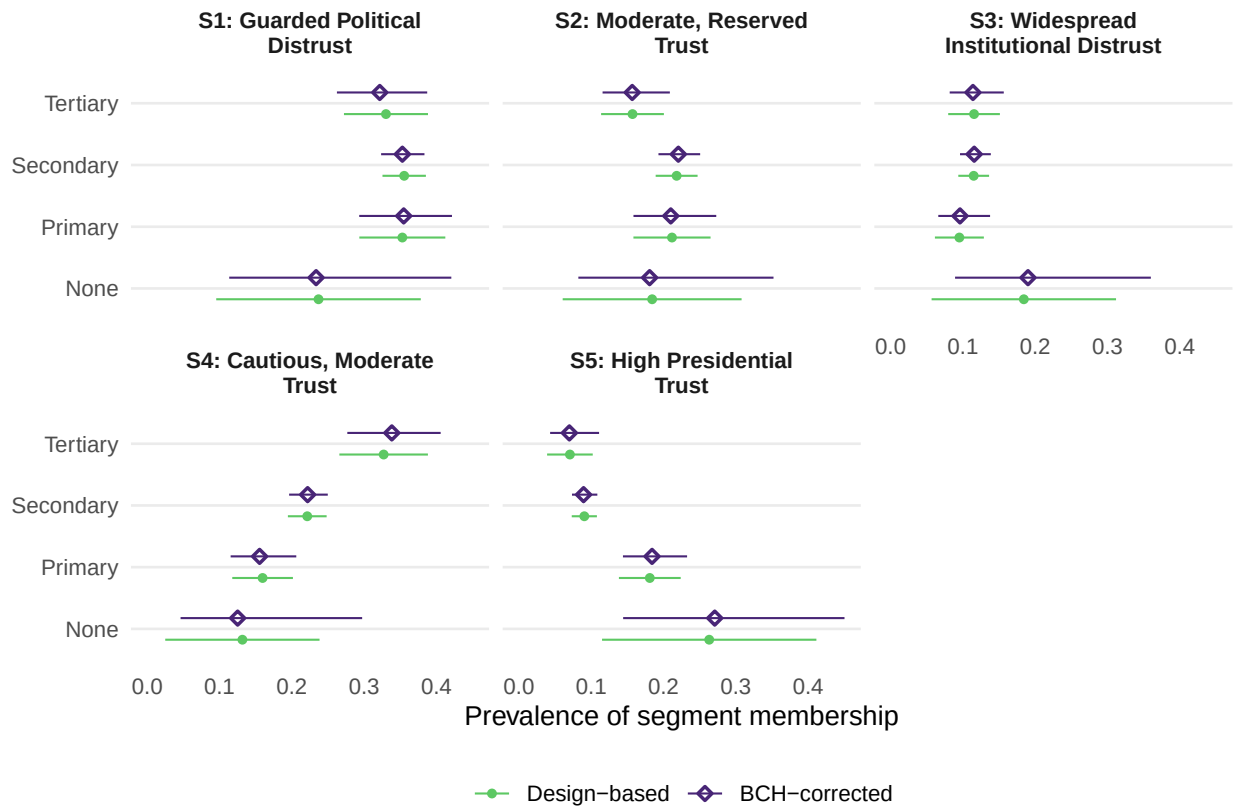
7.3 education

Table 10: Composition by education. Share is the design-weighted proportion among respondents who answered the item, with its JKn standard error.

Level	n	Share
Secondary	940	0.582 (0.015)
None	38	0.024 (0.004)
Primary	326	0.202 (0.012)
Tertiary	312	0.193 (0.015)

Table 11: Segment prevalence within each level of education, uncorrected and BCH-corrected. Rows sum to one across segments. Intervals are in the figure below and the exported CSV.

Level	Estimator	S1	S2	S3	S4	S5
None	Design-based	0.237	0.184	0.184	0.132	0.263
None	BCH-corrected	0.233	0.181	0.190	0.125	0.271
Primary	Design-based	0.353	0.212	0.095	0.160	0.181
Primary	BCH-corrected	0.355	0.210	0.096	0.155	0.184
Secondary	Design-based	0.355	0.218	0.115	0.221	0.090
Secondary	BCH-corrected	0.353	0.220	0.116	0.222	0.089
Tertiary	Design-based	0.330	0.157	0.115	0.327	0.071
Tertiary	BCH-corrected	0.322	0.157	0.114	0.338	0.070



95% intervals from JKn replicate variance and the design t. Filled circles: design-based. Open diamonds: BCH-c

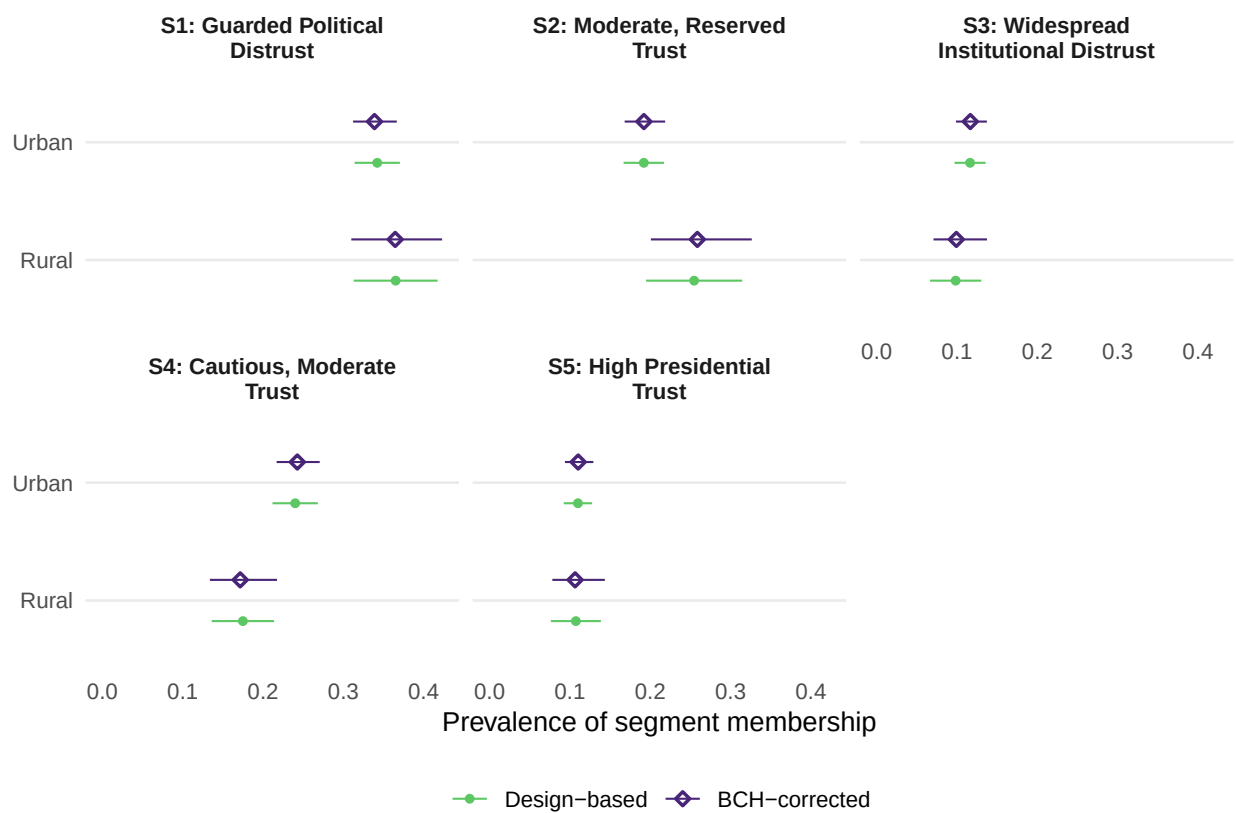
7.4 urban

Table 12: Composition by urban. Share is the design-weighted proportion among respondents who answered the item, with its JK_n standard error.

Level	n	Share
Urban	1292	0.799 (0.035)
Rural	326	0.201 (0.035)

Table 13: Segment prevalence within each level of urban, uncorrected and BCH-corrected. Rows sum to one across segments. Intervals are in the figure below and the exported CSV.

Level	Estimator	S1	S2	S3	S4	S5
Rural	Design-based	0.365	0.255	0.098	0.175	0.107
Rural	BCH-corrected	0.364	0.259	0.099	0.172	0.106
Urban	Design-based	0.342	0.192	0.116	0.240	0.110
Urban	BCH-corrected	0.339	0.192	0.116	0.243	0.110



95% intervals from JK_n replicate variance and the design t. Filled circles: design-based. Open diamonds: BCH-corrected

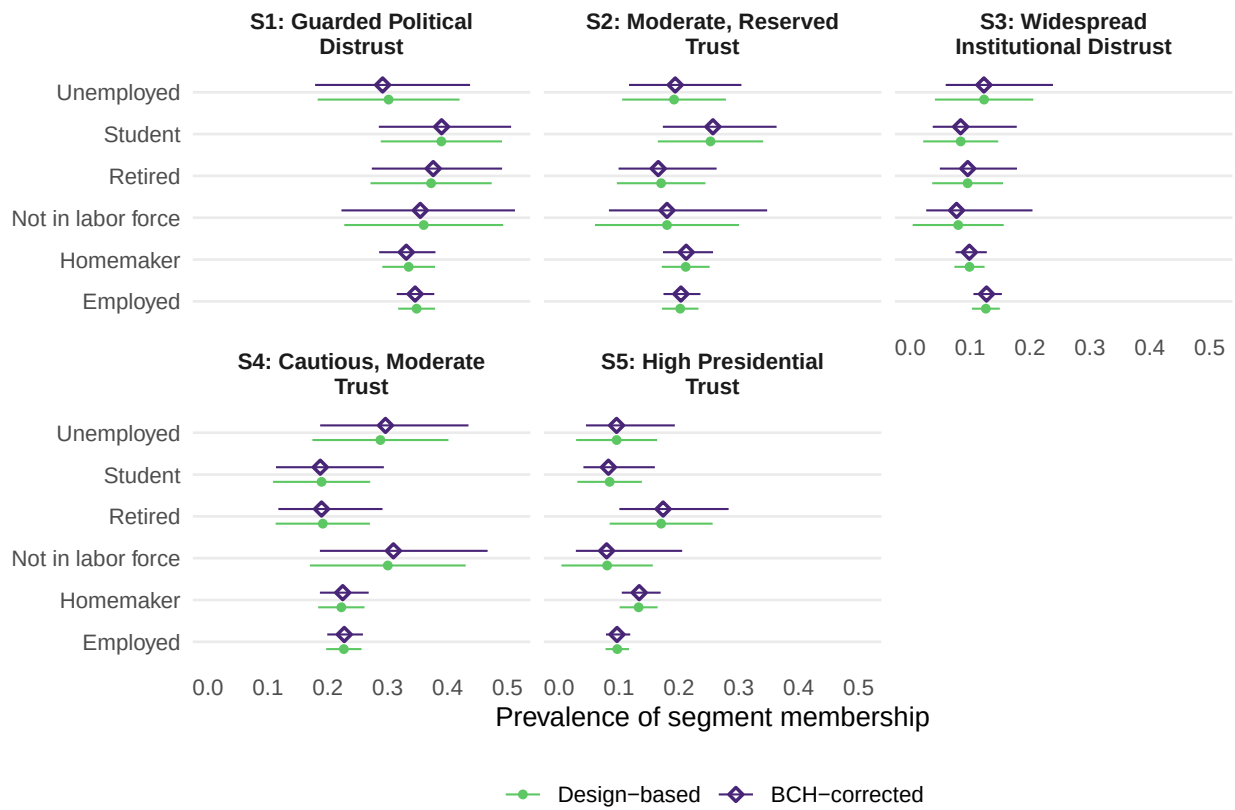
7.5 employment

Table 14: Composition by employment. Share is the design-weighted proportion among respondents who answered the item, with its JK_n standard error.

Level	n	Share
Employed	856	0.531 (0.012)
Unemployed	73	0.045 (0.005)
Not in labor force	50	0.031 (0.004)
Student	95	0.059 (0.006)
Homemaker	445	0.276 (0.012)
Retired	94	0.058 (0.006)

Table 15: Segment prevalence within each level of employment, uncorrected and BCH-corrected. Rows sum to one across segments. Intervals are in the figure below and the exported CSV.

Level	Estimator	S1	S2	S3	S4	S5
Employed	Design-based	0.348	0.202	0.126	0.227	0.097
Employed	BCH-corrected	0.346	0.203	0.127	0.227	0.096
Homemaker	Design-based	0.335	0.211	0.099	0.222	0.133
Homemaker	BCH-corrected	0.331	0.212	0.099	0.225	0.134
Not in labor force	Design-based	0.360	0.180	0.080	0.300	0.080
Not in labor force	BCH-corrected	0.354	0.180	0.077	0.309	0.079
Retired	Design-based	0.372	0.170	0.096	0.191	0.170
Retired	BCH-corrected	0.376	0.165	0.096	0.189	0.173
Student	Design-based	0.389	0.253	0.084	0.189	0.084
Student	BCH-corrected	0.390	0.257	0.084	0.187	0.082
Unemployed	Design-based	0.301	0.192	0.123	0.288	0.096
Unemployed	BCH-corrected	0.291	0.194	0.123	0.296	0.096



95% intervals from JK_n replicate variance and the design t. Filled circles: design-based. Open diamonds: E

7.6 Do the segments differ across demographic levels?

Each level compared against the first level of its variable, within segment, using the replicate covariance so the difference carries its own standard error rather than being read off overlapping intervals. Holm adjustment within each demographic variable.

```
contrasts <- dom_meta |>
  group_by(pvar, segment) |>
  mutate(ref_idx = idx[1], ref_level = plevel[1]) |>
  ungroup() |>
  filter(plevel != ref_level) |>
  mutate(diff = dom_est[idx] - dom_est[ref_idx],
         se = sqrt(dom_V[cbind(idx, idx)] + dom_V[cbind(ref_idx, ref_idx)] -
                   2 * dom_V[cbind(idx, ref_idx)]),
         p_raw = 2 * pt(-abs(diff / se), df = design_df)) |>
  group_by(pvar) |>
  mutate(p_adj = p.adjust(p_raw, method = "holm")) |>
  ungroup()

contrasts |>
  filter(p_adj < 0.05) |>
  arrange(pvar, p_adj) |>
  transmute(Variable = pvar, Comparison = paste(plevel, "vs", ref_level),
            Segment = lab_short(segment),
            Difference = sprintf("%.3f", diff),
            `95% CI` = sprintf("[%+.3f, %+3f]", diff - crit * se, diff + crit * se),
            `Adj. p` = sprintf("%.4f", p_adj)) |>
  lca_table(align = "lllrrr",
            widths = c("6em", "13em", "4em", "6em", "10em", "5em"),
            caption = "Segment composition differences reaching adjusted significance. Each row")
```

Table 16: Segment composition differences reaching adjusted significance. Each row compares one demographic level against that variable's reference level within one segment. An empty table means no level difference survives adjustment.

Variable	Comparison	Segment	Difference	95% CI	Adj. p
age_cat	60+ vs 18-29	S5	+0.075	[+0.026, +0.125]	0.0444
education	Primary vs Secondary	S5	+0.095	[+0.046, +0.143]	0.0028
education	Tertiary vs Secondary	S4	+0.116	[+0.044, +0.189]	0.0270
urban	Rural vs Urban	S4	-0.071	[-0.124, -0.018]	0.0424

8 The handoff

The `.sav` carries the segment assignment, the posterior probabilities, and the BCH correction weights for every scored respondent. The posteriors and correction weights are exported deliberately: cross-tabulating the modal assignment alone attenuates every association by the classification error rate, which is what the BCH tables above correct for.

```

U_full <- bch_weights(P_fixed, modal_fixed, weights(des, "sampling"))
colnames(U_full) <- paste0("bch_segment", seq_len(K_star))

out_sav <- survey_scored |>
  left_join(bind_cols(select(scored, all_of(cfg$id)), as_tibble(U_full)),
    by = cfg$id) |>
  mutate(segment_label = if_else(is.na(segment), NA_character_,
    segment_labels$Label[segment]),
    segment = haven::labelled(
      segment,
      labels = set_names(segment_labels$K, segment_labels$Label),
      label = paste("Segment (design-weighted LCA). Modal assignment:",
        "cross-tabs on this variable alone are attenuated by",
        "classification error. Use the post_segment columns or",
        "the bch_segment weights for unbiased composition.)))

haven::write_sav(out_sav, file.path(cfg$out_dir, "survey_with_segments.sav"))

write_csv(mutate(ci_tbl, segment_label = segment_labels$Label[segment]),
  file.path(cfg$out_dir, "measurement_model_with_CIs.csv"))
write_csv(transmute(domain_all, demographic = variable, level, estimator, segment,
  segment_label = segment_labels$Label[segment],
  prevalence = p, se, lo, hi),
  file.path(cfg$out_dir, "segment_demographics.csv"))
write_csv(transmute(contrasts, demographic = pvar, level = plevel,
  reference = ref_level, segment, difference = diff, se, p_adj),
  file.path(cfg$out_dir, "segment_contrasts.csv"))

cat(sprintf("Wrote survey_with_segments.sav (%d scored) and three CSVs to %s\n",
  nrow(scored), cfg$out_dir))

```

Wrote survey_with_segments.sav (1618 scored) and three CSVs to D:/repos/weighted_and_biased/out